

Assignment 2 — Software Fault-Tolerant Design

AITU Student Scholarship and Tuition Payment Processing

Astana IT University | Fault Tolerance | Academic Year 2026–2027

Context note: All operational data in this assignment are synthetic educational data. They are designed to represent a realistic Kazakhstan/Astana university context and do not describe actual AITU infrastructure, incidents, or internal systems.

Scenario

Consider a synthetic digital service used by an Astana university to process student payment-related transactions. The service must continue processing independent transactions when network, timeout, or database faults occur.

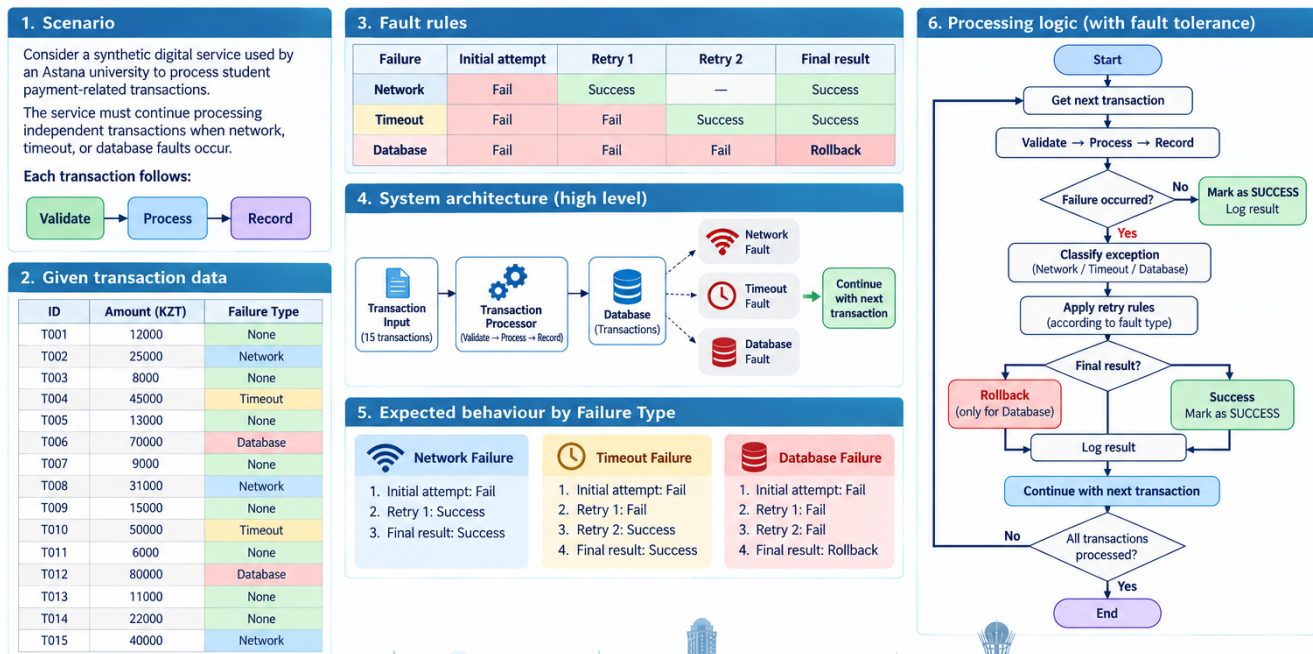
Each transaction follows: Validate → Process → Record.

Assignment 2 — Software Fault-Tolerant Design

AITU Student Scholarship and Tuition Payment Processing

Astana IT University | Fault Tolerance | Academic Year 2026–2027

Context note: All operational data in this assignment are synthetic educational data. They are designed to represent a realistic Kazakhstan/Astana university context and do not describe actual AITU infrastructure, incidents, or internal systems.



Given transaction data

ID	Amount (KZT)	Failure Type
T001	12000	None
T002	25000	Network

T003	8000	None
T004	45000	Timeout
T005	13000	None
T006	70000	Database
T007	9000	None
T008	31000	Network
T009	15000	None
T010	50000	Timeout
T011	6000	None
T012	80000	Database
T013	11000	None
T014	22000	None
T015	40000	Network

Fault rules

Failure	Initial attempt	Retry 1	Retry 2	Final result
Network	Fail	Success	—	Success
Timeout	Fail	Fail	Success	Success
Database	Fail	Fail	Fail	Rollback

Part A — Baseline Implementation (5 points)

Implement a baseline Python processor without fault tolerance. It stops when a failure occurs.

Metric	Result
Transactions attempted	2
Successful transactions	1
Transactions lost	14 (T002 failed + 13 never started)
Processed amount (KZT)	12,000

Metric	Result
Lost amount (KZT)	425,000

Part B — Exception Handling (5 points)

Modify the program so one failed transaction does not terminate the application. Use try/except, failure classification, structured logging, and continuation.

Transaction	Failure	Exception	Recovery Action	Final Status
T002	Network	NetworkError	Caught, logged as transient, skipped. Batch continues.	FAILED (batch continued)
T004	Timeout	GatewayTimeoutError	Caught, logged (charge state unknown), skipped. Batch continues.	FAILED (batch continued)
T006	Database	DatabaseError	Caught, logged, skipped. No rollback in Part B.	FAILED (batch continued)
T008	Network	NetworkError	Caught, logged as transient, skipped. Batch continues.	FAILED (batch continued)
T010	Timeout	GatewayTimeoutError	Caught, logged (charge state unknown), skipped. Batch continues.	FAILED (batch continued)
T012	Database	DatabaseError	Caught, logged, skipped. No rollback in Part B.	FAILED (batch continued)
T015	Network	NetworkError	Caught, logged as transient, skipped. Batch continues.	FAILED (batch continued)

Part C — Retry Mechanism (7 points)

Implement the exact retry policy provided in the input. Generate an attempt log automatically.

Timestamp/Step	Transaction	Attempt	Failure	Action	Result
#01 19:28:35.413	T001	1 (initial)	none	record	SUCCESS
#02 19:28:35.413	T002	1 (initial)	Network	retry in 0.05 s	FAIL
#03 19:28:35.463	T002	2 (retry 1)	none	record	SUCCESS
#04 19:28:35.463	T003	1 (initial)	none	record	SUCCESS

Timestamp/Step	Transaction	Attempt	Failure	Action	Result
#05 19:28:35.463	T004	1 (initial)	Timeout	retry in 0.05 s	FAIL
#06 19:28:35.513	T004	2 (retry 1)	Timeout	retry in 0.10 s	FAIL
#07 19:28:35.614	T004	3 (retry 2)	none	record	SUCCESS
#08 19:28:35.615	T005	1 (initial)	none	record	SUCCESS
#09 19:28:35.617	T006	1 (initial)	Database	retry in 0.05 s	FAIL
#10 19:28:35.667	T006	2 (retry 1)	Database	retry in 0.10 s	FAIL
#11 19:28:35.768	T006	3 (retry 2)	Database	rollback to CP1	ROLLED BACK
#12 19:28:35.769	T007	1 (initial)	none	record	SUCCESS
#13 19:28:35.769	T008	1 (initial)	Network	retry in 0.05 s	FAIL
#14 19:28:35.820	T008	2 (retry 1)	none	record	SUCCESS
#15 19:28:35.820	T009	1 (initial)	none	record	SUCCESS
#16 19:28:35.821	T010	1 (initial)	Timeout	retry in 0.05 s	FAIL
#17 19:28:35.872	T010	2 (retry 1)	Timeout	retry in 0.10 s	FAIL
#18 19:28:35.973	T010	3 (retry 2)	none	record	SUCCESS
#19 19:28:35.974	T011	1 (initial)	none	record	SUCCESS
#20 19:28:35.976	T012	1 (initial)	Database	retry in 0.05 s	FAIL
#21 19:28:36.026	T012	2 (retry 1)	Database	retry in 0.10 s	FAIL
#22 19:28:36.128	T012	3 (retry 2)	Database	rollback to CP2	ROLLED BACK
#23 19:28:36.128	T013	1 (initial)	none	record	SUCCESS
#24 19:28:36.129	T014	1 (initial)	none	record	SUCCESS
#25 19:28:36.129	T015	1 (initial)	Network	retry in 0.05 s	FAIL
#26 19:28:36.180	T015	2 (retry 1)	none	record	SUCCESS

Part D — Checkpoint and Rollback (8 points)

Create a checkpoint after every 5 successfully processed transactions. On a database failure, rollback to the latest valid checkpoint and continue.

Checkpoint	Successful transactions	Total amount (KZT)	State saved
CP1	5 (last: T005)	103,000	success=5, total=103,000, committed T001-T005, in_flight={}, DB rows=5, gateway net=103,000 -> logs/checkpoints/CP1.json
CP2	10 (last: T011)	214,000	success=10, total=214,000, committed T001-T005 + T007-T011, in_flight={}, DB rows=10 -> CP2.json
CP3 (end of batch)	13 (last: T015)	287,000	End of batch (the rule gives only 13 successes). success=13, total=287,000, DB rows=13 -> CP3.json

Implementation architecture

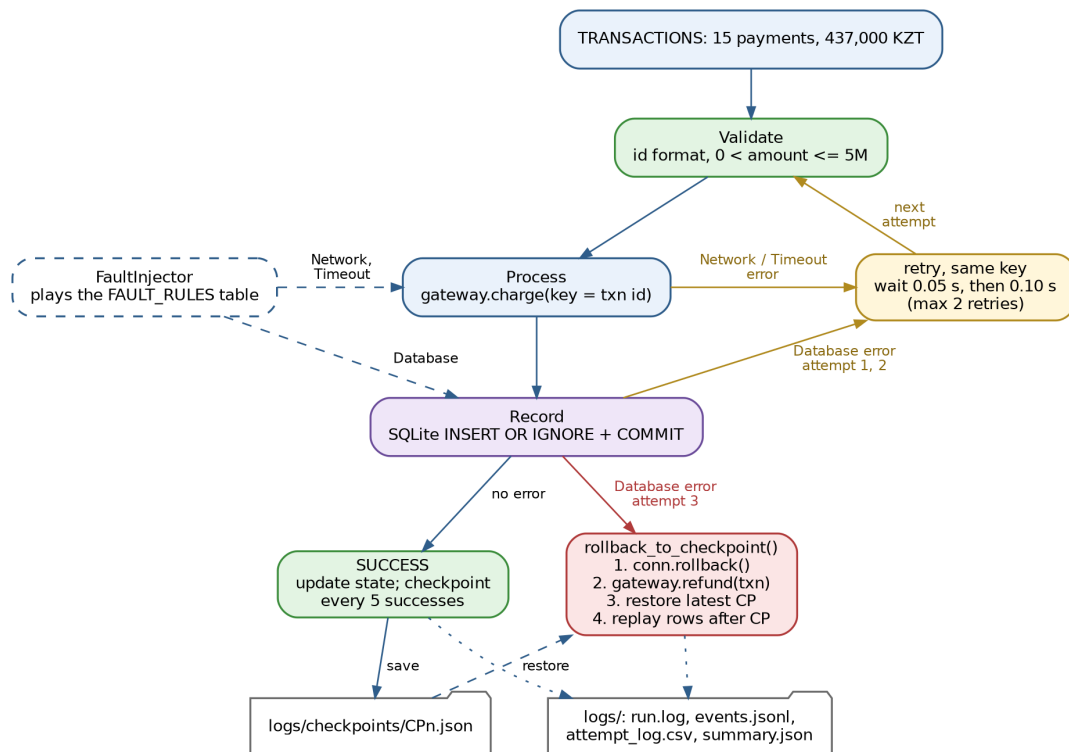


Figure 1. My implementation: one attempt is Validate -> Process -> Record. Network and timeout errors go back to the retry loop with the same key; a database error on the third attempt goes to rollback_to_checkpoint().

Part E — Before/After Analysis (5 points)

Metric	Baseline	Fault-tolerant	Improvement
Successful transactions	1	13	+12 (13x more)

Metric	Baseline	Fault-tolerant	Improvement
Failed transactions	14 (1 failed + 13 not started)	2 (T006, T012 rolled back)	-12 (-85.71%)
Lost amount	425,000 KZT	150,000 KZT	-275,000 KZT (-64.71%)
Retries	0	11	+11 (3 network, 4 timeout, 4 database)
Rollbacks	0	2	+2 (T006 -> CP1, T012 -> CP2)
Completion rate	6.67% (1/15)	86.67% (13/15)	+80.00 pp

1. Calculate recovery improvement.

Completion rate: baseline $1 / 15 = 6.67\%$, fault-tolerant $13 / 15 = 86.67\%$. Improvement = $86.67 - 6.67 = +80.00$ percentage points. Of the 7 transactions with a fault, the baseline recovered 0, and my fault-tolerant run recovered 5 (T002, T004, T008, T010, T015), so the recovery rate is $5 / 7 = 71.4\%$. The other 2 (T006, T012) were rolled back and refunded. At the end of the run the state, the database and the gateway all show 287,000 KZT.

2. Calculate transaction-loss reduction.

Transaction-loss reduction = $(14 - 2) / 14 = 12 / 14 = 85.71\%$. Amount-loss reduction = $(425,000 - 150,000) / 425,000 = 275,000 / 425,000 = 64.71\%$. The amount reduction is smaller because the two rolled-back payments are big: 70,000 and 80,000 KZT.

3. Which mechanism contributed most?

I used the Part B run as a middle step to see what each mechanism adds. Exception handling: 1 -> 8 successful (+7 transactions, +84,000 KZT). Retry: 8 -> 13 (+5 transactions, +191,000 KZT). Rollback: +0 transactions. By count, exception handling helps more. By money, retry helps more, because the faulty payments are bigger ($341,000 / 7 = 48,714$ KZT on average, clean ones $96,000 / 8 = 12,000$ KZT). I think retry contributed most, because it is the only mechanism that turns a failed payment into a successful one. Rollback adds no successful payments. Its job is to keep the money and the records equal.

4. When is retry unsafe or inappropriate?

Retry is unsafe when the operation is not idempotent and the result is unknown. A timeout is exactly this case: in my gateway the money is taken, and only the answer is lost. When I ran the same batch without idempotency keys, T004 and T010 were charged 3 times each, 190,000 KZT more than the database recorded. Retry is also useless for permanent errors, for example a wrong amount or a blocked account: my `ValidationError` has `retryable=False` and is never retried. And retry needs a limit and a pause (backoff), otherwise many clients repeat requests at the same moment and overload a service that is already slow.

5. Why is rollback required for database failure?

At the moment of the database error the Process step is already done: the student's money is taken, but the payment row is not saved. This is a half-finished transaction. In Part B (no rollback) the failed

INSERT stayed in the open SQLite transaction and was saved together with the next commit, so T006 and T012 are in the DB but marked FAILED. The three totals did not match: state 96,000, DB 246,000, gateway 341,000 KZT. Retry cannot fix this, because the rules say the database fails 3 times. Rollback removes the uncommitted row, refunds the charge, and restores the last checkpoint, so the three totals are equal again.

6. Why is idempotency important in student payment processing?

Scholarship and tuition payments are real money of real students. If a timeout happens and the system just repeats the request, the student can pay twice and then has to ask for a refund. In my code the transaction ID is the idempotency key in three places: the gateway ignores a repeated charge with the same key, SQLite has `txn_id` as PRIMARY KEY with INSERT OR IGNORE, and the processor skips IDs it already recorded. Because of this, running the same batch a second time is safe: the 13 recorded payments are skipped, T006 and T012 go through, and the total is 437,000 KZT with no double charge (`test_second_run_of_same_batch_does_not_charge_again`).

Short technical conclusion

The baseline processed 1 of 15 payments (12,000 of 437,000 KZT) and stopped at T002. With exception handling, retry and checkpoint/rollback, the same data gives 13 successful payments (287,000 KZT), 11 retries and 2 rollbacks. The completion rate goes from 6.67% to 86.67%. For payments the program must survive the error and also keep the money and the records equal. In Part B the program kept running but ended with three different totals. With retry I needed an idempotency key, and with the database failure I needed a rollback that also refunds the charge. After that, all three totals were 287,000 KZT.

Submission

- Python source code
- Generated transaction and retry logs
- Completed tables
- Before/after analysis
- Short technical conclusion
- Oral defense

STUDENT TECHNICAL WORK REPORT / SUBMISSION RECORD

Assignment 2 — Software Fault-Tolerant Design

Purpose: This form is a formal submission record. The student must complete every applicable field and attach evidence of their own work. Short generic explanations are not sufficient.

1. Student and Submission Information

Required field	Student entry
Student full name	Alibek Asset
Student ID	255175
Group / cohort	CSE-2503
Instructor	Azamat Serek
Date of submission	27.09.2026
Git repository	https://github.com/Gontafi/university-stuff/tree/FTR-assignment-02
Attached code / file names	payment_processor.py, test_payment_processor.py, logs/ (run.log, events.jsonl, attempt_log.csv, checkpoints/CP0-CP3.json, summary.json, tests.txt)

2. Work Period and Task Record

Item	Student entry
Work start date/time	27.09.2026, 19:20
Work completion date/time	27.09.2026, 19:33
Main task completed	Parts A-E: baseline processor, exception handling, retry policy from the fault rules, checkpoint and rollback, before/after analysis. Plus 12 unit tests and one extra experiment without idempotency keys.
Tools / technologies used	Python 3.12.3, standard library only (sqlite3, logging, json, csv, unittest). Graphviz for the architecture diagram. Run in the Linux terminal.
Main technical output produced	payment_processor.py prints all tables and writes the logs on every run: run.log, events.jsonl (structured), attempt_log.csv (26 attempts), checkpoints CP0-CP3 as JSON, summary.json with every number used in this report.

3. Report Structure to Complete

For each section below, write the requested evidence in your own words. Include calculations, diagrams, code references, screenshots, logs, or experimental evidence where applicable.

1. Baseline Implementation

Describe your baseline processor, the stopping behavior after a failure, and provide the required metrics from your own execution.

Required content	Student response / evidence
Summary	run_baseline() calls attempt() (Validate -> Process -> Record) for every transaction with no try/except. T001 passes. T002 raises NetworkError in the Process step, nothing catches it, and the program stops. Result: 2 attempted, 1 successful, 14 lost (T002 failed + 13 never started), 12,000 KZT processed, 425,000 KZT lost. That is 97.3% of the 437,000 KZT batch.
Evidence / calculation / artifact reference	payment_processor.py: run_baseline(), part_a(). logs/run.log, line "baseline_crash". The command python3 payment_processor.py --crash shows the real traceback: NetworkError: T002 Process: connection reset, request did not reach the gateway. Test: BaselineTest.test_stops_at_first_failure.
Student-specific decision or observation	The baseline uses the same attempt() function as the fault-tolerant version, so the only difference between the runs is the error handling, and the comparison in Part E is fair. I count the 13 transactions after T002 as lost too, because nobody processes them until someone restarts the program by hand.

2. Exception Handling

Explain your exception classes/handling logic and show how processing continues after an individual transaction failure.

Required content	Student response / evidence
Summary	All errors inherit from PaymentError, which has two fields: kind and retryable. The subclasses are ValidationError (not retryable), NetworkError, GatewayTimeoutError and DatabaseError. process() wraps each attempt in try/except PaymentError, writes a JSON event, marks the transaction FAILED and goes to the next one. Result: 8 successful, 7 failed, 96,000 KZT processed, and the batch finished all 15 transactions.
Evidence / calculation / artifact reference	logs/events.jsonl (one JSON object per event, with txn, attempt, exception, retryable, error). Part B table above. Tests: test_batch_continues_after_failures, test_validation_error_is_not_retried.

Required content	Student response / evidence
Student-specific decision or observation	I called the timeout class GatewayTimeoutError, not TimeoutError, so it does not hide the built-in Python TimeoutError. I also added a consistency check at the end of the run. For Part B it printed MISMATCH: state 96,000, DB 246,000, gateway 341,000 KZT. So with exception handling alone the program keeps running, but the money and the records do not match.

3. Retry Mechanism

Document the exact retry policy, generate the attempt log, and explain how the implementation follows the provided network/timeout/database rules.

Required content	Student response / evidence
Summary	Policy: at most 2 retries (3 attempts), backoff 0.05 s and then 0.10 s (scaled down from 1 s and 2 s so the run is fast). Only errors with retryable=True are retried. FaultInjector plays the fault rules table: Network fail/success, Timeout fail/fail/success, Database fail/fail/fail. Result: 26 attempts and 11 retries (3 network x 1, 2 timeout x 2, 2 database x 2). 5 transactions were recovered.
Evidence / calculation / artifact reference	logs/attempt_log.csv (26 rows, written by the program, not by hand). Part C table above. Test: RetryTest.test_attempts_follow_fault_rules checks 2 attempts for T002, 3 for T004 and T006, and 11 retries in total.
Student-specific decision or observation	I retry the whole Validate -> Process -> Record chain, not only the failed step, so Process runs again on every retry. To make this safe, the transaction ID is the idempotency key in the gateway and the PRIMARY KEY in SQLite. I also made the gateway timeout happen after the money is taken (only the answer is lost), because in a real bank this is the case that causes double payments.

4. Checkpoint and Rollback

Show where checkpoints are created, what state is saved, and how rollback works after database failure.

Required content	Student response / evidence
Summary	A checkpoint is made when success % 5 == 0, plus CP0 at the start and a final one at the end of the batch. Saved state: success count, total, committed IDs, in-flight payments, DB row count and gateway net amount (deep copy + JSON file). On DatabaseError after the retries, rollback_to_checkpoint() does 4 things: conn.rollback() drops the uncommitted INSERT, gateway.refund() returns the money, the state is restored from the latest checkpoint, and rows committed after the checkpoint are replayed from the DB. T006 -> CP1: 173,000 -> 103,000, refund 70,000. T012 -> CP2: 294,000 -> 214,000, refund 80,000.

Required content	Student response / evidence
Evidence / calculation / artifact reference	logs/checkpoints/CP0.json - CP3.json. The "rollback" events in events.jsonl (dirty_total, restored_total, refunded, replayed, consistent=True). Tests: test_checkpoints_every_five_successes, test_rollback_keeps_state_db_and_gateway_equal, test_rollback_replays_rows_committed_after_checkpoint.
Student-specific decision or observation	The rule "every 5 successful" gives only CP1 and CP2, because only 13 payments succeed. I added a final checkpoint (CP3) so T013-T015 are saved too. I also noticed that both database failures come right after a checkpoint (T006 after CP1, T012 after CP2), so here the rollback loses nothing (replayed = 0). To check the general case, I wrote a test where the DB fails 3 transactions after a checkpoint, and the 2 committed rows are replayed correctly.

5. Before/After Analysis

Compare baseline and fault-tolerant execution using the required metrics. Show calculations for recovery improvement and transaction-loss reduction.

Required content	Student response / evidence
Summary	Successful: 1 -> 13. Failed: 14 -> 2. Lost amount: 425,000 -> 150,000 KZT. Retries: 0 -> 11. Rollbacks: 0 -> 2. Completion: 6.67% -> 86.67% (+80.00 pp). Transaction-loss reduction $(14 - 2) / 14 = 85.71\%$. Amount-loss reduction $(425,000 - 150,000) / 425,000 = 64.71\%$.
Evidence / calculation / artifact reference	Part E table above. logs/summary.json, key part_e. The same table is printed at the end of logs/run.log.
Student-specific decision or observation	I added Part B as a middle column (8 successful, 341,000 KZT lost), so I can see what each mechanism adds. Exception handling gives +7 payments but only +84,000 KZT. Retry gives +5 payments but +191,000 KZT. The failed payments are about 4 times bigger than the clean ones on average, so counting only transactions would undervalue retry.

6. Technical Reflection

Explain retry safety, rollback, and idempotency specifically in the context of your implementation.

Required content	Student response / evidence
Summary	Network errors happen before the request reaches the gateway, so repeating them is safe. With a timeout the charge is already done, and I can repeat it only because of the idempotency key. Database errors get 2 retries, then rollback with a refund. The key is checked in the gateway, in SQLite (PRIMARY KEY with INSERT OR IGNORE) and in the processor, which skips IDs that are already recorded.

Required content	Student response / evidence
Evidence / calculation / artifact reference	Extra run in run.log ("WITHOUT idempotency keys"): gateway charged 477,000 KZT, DB recorded 287,000 KZT, overcharge 190,000 KZT (T004 x3, T010 x3). Tests: test_keys_prevent_double_charge_after_timeout, test_second_run_of_same_batch_does_not_charge_again.
Student-specific decision or observation	A rolled-back payment can be sent again later. When I ran the same batch a second time on the same processor, the 13 recorded payments were skipped, T006 and T012 went through (the database was back), and the total became 437,000 KZT. Nobody was charged twice.

4. Individual Work and Anti-Plagiarism Record

Because the assignment uses a common educational scenario and dataset, the submitted report must contain verifiable evidence of the individual student's work. Copying another student's wording, calculations, architecture, code, logs, screenshots, or conclusions is not acceptable.

- Submit a run-generated log, not a manually typed log.
- Include the exact source filename and commit/version used for the reported results.
- Record at least one implementation decision that differs from the simplest possible solution.
- Include one failed test/debugging issue that you personally resolved.
- During oral defense, be ready to modify or explain one recovery rule in your code.

Individual evidence	Student entry
My unique technical decision	I added a three-way consistency check. After every rollback and at the end of each run, the program compares the processor total, the SQLite total and the gateway net amount. I also made the gateway timeout happen after the money is taken, and ran the batch without idempotency keys to measure the overcharge (190,000 KZT).
My own implementation / calculation evidence	payment_processor.py (v2.0, about 640 lines) and test_payment_processor.py (12 tests, all pass, logs/tests.txt). I wrote both and ran them on 27.09.2026. All tables in this report come from logs/summary.json of that run.

Individual evidence	Student entry
Exact file, code, commit, notebook, or log reference	Git commit 8bd7c49 on branch FTR-assignment-02 (code, tests and logs). payment_processor.py v2.0, SHA-256 fa5108bf4a074599cab3a5c51995825fa8c5a3810e05dd81e4d0fcc1abf2f74a (printed in the first line of run.log). Functions: run_baseline(), process(), give_up(), make_checkpoint(), rollback_to_checkpoint(), part_a() - part_e(). Logs: logs/run.log, events.jsonl, attempt_log.csv, checkpoints/, summary.json. Run started 2026-09-27 19:28:35.
One problem I personally encountered and how I solved it	In Part B the DB had 10 rows while only 8 payments were successful. I found that Python sqlite3 opens a transaction on INSERT. When the commit of T006 failed, the INSERT stayed open, and the commit of T007 saved it too. The same happened with T012 and T013. I fixed it in the rollback: conn.rollback() is the first thing rollback_to_checkpoint() does, and I added the consistency check. Now after both rollbacks the DB has exactly 13 rows and 287,000 KZT. The test test_without_rollback_failed_rows_leak_into_db keeps this problem visible.
One limitation of my own solution	The faults are simulated and deterministic: FaultInjector always follows the table, so the numbers are reproducible but there is no random behaviour. The backoff is scaled down to 0.05 s and 0.10 s. Checkpoints are written to JSON, but the program cannot restart from a JSON file if the process itself crashes; it only uses the copy in memory. I also assume that a refund always works; a real refund can fail too and would need its own retry.

5. Evidence Checklist

- ☒ Completed required tables and calculations
- ☒ Architecture / RBD / FTA / security diagram where applicable
- ☒ Source code or configuration files where applicable
- ☒ Execution logs / experiment results where applicable
- ☒ Screenshots or other reproducible evidence where applicable
- ☒ Links to code, datasets, notebooks, or artifacts
- ☒ Before/after comparison where applicable
- ☒ Final technical conclusion

6. Student Declaration

I confirm that this report describes my own work. I have not copied another student's report, code, calculations, diagrams, logs, screenshots, or conclusions. Any external source or reused material has been identified. I understand that I may be asked to reproduce or explain any part of this submission during the oral defense.

Role	Name	Signature	Date
Student / Executor	Alibek Asset		27.09.2026
Instructor / Accepted by	Azamat Serek		_____

7. Instructor Acceptance Record

Check	Status / comment
Report structure completed	☐ Yes ☐ No Comment: _____
Evidence is sufficient	☐ Yes ☐ No Comment: _____
Individual contribution is identifiable	☐ Yes ☐ No Comment: _____
Oral defense completed	☐ Yes ☐ No Comment: _____
Final status	☐ Accepted ☐ Revision required

Appendix A. Source code excerpt (payment_processor.py)

Retry loop, final decision after the retries, and rollback. Full file: *payment_processor.py* (v2.0, SHA-256 *fa5108bf4a074599...*).

```
def process(self, txn):
    self.attempted += 1
    if txn.id in self.state["committed"]:
        self.event(logging.INFO, "txn_skipped", txn=txn.id, reason="already recorded")
        self.status[txn.id] = "SKIPPED (already recorded)"
        return self.status[txn.id]

    max_attempts = 1 + (MAX_RETRIES if self.retries else 0)
    for n in range(1, max_attempts + 1):
        try:
            self.attempt(txn)
        except PaymentError as err:
            if err.retryable and n < max_attempts:
                delay = self.backoff * 2 ** (n - 1)
                self.retry_count += 1
                self.log_attempt(txn, n, err, f"retry in {delay:.2f} s", "FAIL")
                time.sleep(delay)
                continue
            return self.give_up(txn, n, err)
        self.log_attempt(txn, n, None, "record", "SUCCESS")
        self.status[txn.id] = "SUCCESS"
        if self.use_checkpoints and self.state["success"] % CHECKPOINT_EVERY == 0:
            self.make_checkpoint(f"{self.state['success']} successful transactions")
        return "SUCCESS"

def give_up(self, txn, n, err):
    if isinstance(err, DatabaseError) and self.use_checkpoints:
        cp = self.rollback_to_checkpoint(txn)
        self.log_attempt(txn, n, err, f"rollback to {cp}", "ROLLED BACK")
        self.status[txn.id] = "ROLLED BACK"
    else:
        if not err.retryable:
            action = "not retryable, skip"
        elif self.retries:
            action = "retries used up, skip"
        else:
            action = "skip, continue batch"
        self.log_attempt(txn, n, err, action, "FAILED")
        self.status[txn.id] = "FAILED"
    return self.status[txn.id]

def rollback_to_checkpoint(self, txn):
    cp = self.checkpoints[-1]
    dirty_total = self.state["total"] + sum(self.state["in_flight"].values())
    self.db.rollback() # drop the uncommitted INSERT
    refunded = self.gateway.refund(txn.id) # give the money back to the student
    restored = copy.deepcopy(cp["state"])
    # rows committed after the checkpoint are replayed from the DB (source of truth)
    rows = self.db.rows()
    replayed = [t for t in self.state["committed"] if t not in restored["committed"]]
    for t in replayed:
        restored["committed"].append(t)
        restored["success"] += 1
        restored["total"] += rows[t]
    self.state = restored
    self.rollback_count += 1
    self.event(logging.WARNING, "rollback", txn=txn.id, to=cp["name"], dirty_total=dirty_total,
        restored_total=self.state["total"], refunded=refunded, replayed=len(replayed),
        consistent=self.consistency()["ok"])
    return cp["name"]
```

Appendix B. Run-generated log excerpt (logs/run.log)

Run started 2026-09-27 19:28:35. Lines are copied from *logs/run.log* without changes; the full log is attached.

```
payment_processor.py v2.0 sha256 fa5108bf4a074599 Python 3.12.3
Run started 2026-09-27T19:28:35 15 transactions, 437,000 KZT
=== Part A - Baseline (no fault tolerance) ===
19:28:35.411 ERROR baseline_crash mode=baseline exception=NetworkError error=T002 Process: connection reset, request did
not reach the gateway attempted=2
=== Part B - Exception handling (no retry, no rollback) ===
Consistency check: state total 96,000, DB total 246,000, gateway charged 341,000 -> MISMATCH
Rows of failed transactions that still reached the DB (no rollback): T006, T012
=== Parts C and D - Retry + checkpoint/rollback (fault-tolerant run) ===
19:28:35.413 INFO checkpoint mode=fault-tolerant cp=CP0 success=0 total=0 last_txn=None reason=start of batch
19:28:35.616 INFO checkpoint mode=fault-tolerant cp=CP1 success=5 total=103000 last_txn=T005 reason=5 successful
transactions
```

```

19:28:35.768 WARNING rollback                mode=fault-tolerant txn=T006 to=CP1 dirty_total=173000 restored_total=103000
refunded=70000 replayed=0 consistent=True
19:28:35.975 INFO    checkpoint              mode=fault-tolerant cp=CP2 success=10 total=214000 last_txn=T011 reason=10 successful
transactions
19:28:36.127 WARNING rollback                mode=fault-tolerant txn=T012 to=CP2 dirty_total=294000 restored_total=214000
refunded=80000 replayed=0 consistent=True
19:28:36.181 INFO    checkpoint              mode=fault-tolerant cp=CP3 success=13 total=287000 last_txn=T015 reason=end of batch
Consistency check: state total 287,000, DB total 287,000, gateway charged 287,000 -> OK
=== Part E - Before/after ===
Transaction-loss reduction: (14 - 2) / 14 = 85.71%
Amount-loss reduction: (425,000 - 150,000) / 425,000 = 64.71%
Recovered by retry: 5 of 7 faulty transactions (T002, T004, T008, T010, T015)
=== Extra - same fault-tolerant run WITHOUT idempotency keys ===
Gateway charged 477,000 KZT, DB recorded 287,000 KZT, overcharge 190,000 KZT
T004: charged 3 times x 45,000 KZT
T010: charged 3 times x 50,000 KZT

```

Appendix C. Unit test results (logs/tests.txt)

```

test_stops_at_first_failure ... ok
test_checkpoint_is_not_changed_by_later_work ... ok
test_checkpoints_every_five_successes ... ok
test_rollback_keeps_state_db_and_gateway_equal ... ok
test_rollback_replays_rows_committed_after_checkpoint ... ok
test_batch_continues_after_failures ... ok
test_validation_error_is_not_retried ... ok
test_without_rollback_failed_rows_leak_into_db ... ok
test_keys_prevent_double_charge_after_timeout ... ok
test_second_run_of_same_batch_does_not_charge_again ... ok
test_attempts_follow_fault_rules ... ok
test_final_status ... ok

```

```

-----
Ran 12 tests in 0.010s

```

```

OK

```